

RL 最终报、最本质的数学目标是优化 Total Return, 即累积奖励, Surrogate objectives 则是 ^{直接优化 Return 方差很大} 为提供稳定梯度, ^{否则训练大量与环境交互} 避免重新采样等原因而设计的代理目标,

而 V、Q、A 子都是构建 surrogate 的辅助工具

gradient estimator: $\hat{g} = \hat{E}_t[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \hat{A}_t]$

objective $L^{\pi}(\theta) = \hat{E}_t[\log \pi_{\theta}(a_t|s_t) \hat{A}_t]$

这里的 Estimator 指用近似来计算无法直接观测的量 (如回报、优势子) 它是基于有限样本的估计

如 MC、TD、GAE、n-steps return 都是种估计 (而非真的 Q or Advantage)

↓
无偏、方差大
↓
有偏、方差小
↓
平衡 bias 和 var.

○ 策略梯度算法:

$J(\theta) = E_{s_0} [V^{\pi_{\theta}}(s_0)]$ ← 理论, 损失恒成立.

total return 是初始状态分布 s_0 下的价值函数 $V(s_0)$

层次	名称	定义	是否为 surrogate
L0	性能指标 (Performance Metric)	$\eta(\pi) = \mathbb{E}_{\tau \sim \pi}[R(\tau)]$ (策略的期望累积回报)	✗ 不是 这是唯一终极目标
L1	价值函数 (Value Function)	$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[G_t s_t = s]$ (理论定义, 非近似)	✗ 不是 这是理论工具, 与 $\eta(\pi)$ 等价
L2	Critic 网络 (Function Approximator)	$V_{\theta}(s)$ (神经网络对 V^{π} 的逼近)	✓ 是 这是对 V^{π} 的函数逼近
L3	Surrogate Objective (优化目标)	$L^{CPI}(\theta) = \mathbb{E}[\pi_{\theta}(a_t) \hat{A}_t]$ (用于高效优化的代理目标)	✓ 是 这是对 $\eta(\pi)$ 的优化代理

采样 $\sum_{s \in S} V^{\pi_{\theta}}(s) \sum_{a \in A} Q^{\pi_{\theta}}(s, a) \pi_{\theta}(a|s)$
↓
状态访问分布 ↓
策略网络概率分布

$\nabla_{\theta} J_{\theta} = \sum_{s \in S} V^{\pi_{\theta}}(s) \sum_{a \in A} Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \pi_{\theta}(a|s)$

理论上说 $Q^{\pi_{\theta}}$ 、 $V^{\pi_{\theta}}$ 与 θ 有关系, 但求梯度只对 $\pi_{\theta}(a|s)$ (也就是策略网络) 求。

$= E_{\pi_{\theta}}[Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$ 这里的期望 E 是对 π_{θ} 下的 (s, a) 对而取的。

$= E_{\pi_{\theta}}[Q^{\pi_{\theta}}(s_t, a_t) \cdot E[\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) | s_t, a_t]]$

$$= E_{\pi_0}[\underbrace{Q^{\pi_0}(s_t, a_t)}_{\text{TD target}} | s_t, a_t] \cdot E[V_0 \log \pi_0(a|s) | s_t, a_t]]$$

故而只需 $E_{\pi_0}(\psi^{\pi_0} | s_t, a_t) = E_{\pi_0}[Q^{\pi_0} | s_t, a_t]$,

则:

$$\nabla_{\theta} J_{\theta} = E_{\pi_0}[\psi^{\pi_0}(s_t, a_t) \nabla_{\theta} \log \pi_0(a|s)]$$

故而 $Q(s, a)$ 可换成 MC: $\sum_{t=0}^{\infty} \gamma^t r_t$ 或 $\underbrace{A^{\pi_0}(s, a)}_{Q^{\pi_0} - V^{\pi_0}(s_t)} = r_t + \gamma V^{\pi_0}(s_{t+1}) - V^{\pi_0}(s_t)$ 子。

注意 $E[Q(s, a)] \neq E[A(s, a)]$, 优势函数与 TD 目标可以作 ψ 是因为它们的后一项 $V^{\pi_0}(s_t)$ 不随 $\pi_0(a|s)$ 而变化。

$$\nabla_{\theta} J_{\theta} = E_{\pi_0}[A^{\pi_0}(s, a) \nabla_{\theta} \log \pi_0(a|s)] = E_{\pi_0}[Q^{\pi_0} \nabla_{\theta} \log \pi_0(a|s)] - \underbrace{E_{\pi_0}[V^{\pi_0} \nabla_{\theta} \log \pi_0(a|s)]}_{\text{"} V^{\pi_0} \nabla E_{\pi_0}[\log \pi_0(a|s)] = 0.}$$

再看我们最爱的 TRPO.

PG 如果有约束, 那么基线约束, 初无问题.

而 PO 则需防止代理目标偏离

策略优化与策略梯度的不同就在于优化考虑了限制, 并且前述的策略梯度估计无偏, 下述则有偏 (分布不同)

由于策略优化的推导基于:

$$J(\theta') - J(\theta) = \underbrace{E_{\pi_0}}_{\text{在 } \pi_0 \text{ (旧策略) 的分布中.}} \left[\sum_{t=0}^{\infty} \gamma^t [r(s_t, a_t) + \gamma V^{\pi_0}(s_{t+1}) - V^{\pi_0}(s_t)] \right]$$

$$= \frac{1}{1-\gamma} E_{\pi_0} E_{\pi_0} [A^{\pi_0}(s, a)]$$

但在更新之前 π_0 的分布是不可知的, 所以实际上是用 当前 (旧) 策略 π_0 的分布来计算这个值。

真实性能是: $J(\theta')$

而 Surrogate 是: $L_{\theta}(\theta') = J(\theta) + \frac{1}{1-\gamma} \underbrace{E_{\pi_0} E_{\pi_0}}_{\text{只是按当前分布用旧的}} [A^{\pi_0}(s, a)]$

$$= J(\theta) + E_{\pi_0} E_{\pi_0} \left[\frac{\pi_0'(a|s)}{\pi_0(a|s)} A^{\pi_0}(s, a) \right] \text{ 重要性采样.}$$

俺也不知 $\frac{1}{1-\gamma}$ 上哪去了



我们可以知道新参数下的策略 π_0' .

但无法对每一个可能的 π_0' 都去获得 $V^{\pi_0'}$.

因为这需要拿着无数个 θ' 去跑、去采样

→ 来自 2002 年的 CPI 理论.

TRPO的理论基石是公式 $\eta(\pi') \geq \underbrace{L_{\pi}(\pi')}_{\text{Surrogate}} - \underbrace{C \sqrt{\max_s D_{KL}(\pi||\pi')}}_{\text{分布漂移的代价}}$

真实性能

TRPO量化了由新旧策略不同带来的分布漂移损失的上限，以及限制了策略性能大幅下跌。

优化目标: $\max_{\theta'} L_{\theta}(\theta')$

s.t. $E_{\pi_{\theta^0}} [D_{KL}(\pi_{\theta^0}, \pi_{\theta'})] \leq \delta$

注意这里不要被 $J(\theta') - J(\theta)$ 的推导给先入为主了而一直想怎么找到两者的差值。

实际上我们的优化目标就是 $L_{\theta}(\theta') = J(\theta) + E_{\pi_{\theta^0}} E_{\pi_{\theta^0}} [\frac{\pi_{\theta'}(a|s)}{\pi_{\theta^0}(a|s)} A^{\pi_{\theta^0}}(s,a)]$

θ' 是待改进的参数， $L_{\theta}(\theta')$ 的变量就是 θ' 而非 θ ，所以优化方法就是算 $L_{\theta}(\theta')$ 对 θ' 的梯度，也就是 $\text{ratio} \cdot \text{Advantage}$ 的梯度。

而在标准实现中， π^0, A^0 都是 detach 的。

```
def update(self, transition_dict):
    states = torch.tensor(transition_dict['states'],
                           dtype=torch.float).to(self.device)
    actions = torch.tensor(transition_dict['actions']).view(-1, 1).to(
        self.device)
    rewards = torch.tensor(transition_dict['rewards'],
                           dtype=torch.float).view(-1, 1).to(self.device)
    next_states = torch.tensor(transition_dict['next_states'],
                               dtype=torch.float).to(self.device)
    dones = torch.tensor(transition_dict['dones'],
                          dtype=torch.float).view(-1, 1).to(self.device)
    td_target = rewards + self.gamma * self.critic(next_states) * (1 -
                                                                    dones)

    td_delta = td_target - self.critic(states)
    advantage = compute_advantage(self.gamma, self.lmbda,
                                   td_delta.cpu()).to(self.device)
    old_log_probs = torch.log(self.actor(states).gather(1,
                                                         actions)).detach()
    old_action_dists = torch.distributions.Categorical(
        self.actor(states).detach())
    critic_loss = torch.mean(
        F.mse_loss(self.critic(states), td_target.detach()))
    self.critic_optimizer.zero_grad()
    critic_loss.backward()
    self.critic_optimizer.step() # 更新价值函数
    # 更新策略函数
    self.policy_learn(states, actions, old_action_dists, old_log_probs,
                      advantage)
```

来自策略集的数据

→ GAE 不计算 actor 计

```
def policy_learn(self, states, actions, old_action_dists, old_log_probs,
                 advantage): # 更新策略函数
    surrogate_obj = self.compute_surrogate_obj(states, actions, advantage,
                                                old_log_probs, self.actor)
    grads = torch.autograd.grad(surrogate_obj, self.actor.parameters())
    obj_grad = torch.cat([grad.view(-1) for grad in grads]).detach()
    # 用共轭梯度法计算 x = H^(-1)g
    descent_direction = self.conjugate_gradient(obj_grad, states,
                                                old_action_dists)

    Hd = self.hessian_matrix_vector_product(states, old_action_dists,
                                             descent_direction)
    max_coef = torch.sqrt(2 * self.kl_constraint /
                           (torch.dot(descent_direction, Hd) + 1e-8))
    new_para = self.line_search(states, actions, advantage, old_log_probs,
                                old_action_dists,
                                descent_direction * max_coef) # 线性搜索
    torch.nn.utils.convert_parameters.vector_to_parameters(
        new_para, self.actor.parameters()) # 用线性搜索后的参数更新策略

def compute_surrogate_obj(self, states, actions, advantage, old_log_probs,
                          actor): # 计算策略目标
    log_probs = torch.log(actor(states).gather(1, actions))
    ratio = torch.exp(log_probs - old_log_probs)
    return torch.mean(ratio * advantage)
```

只有这里“ π^0 ”的部分带梯度，即 $\pi^0(s)$ 和 $\pi^0(s)$ 是由同一个 actor 算出来的，但 π^0 有梯度，其 θ 在此时与 θ 是一样的，但可以更新。

注：在 TRPO 中是在 π^0 中会进行多次更新，在一个 epochs 中 actor update 了多次， π^0 与 π 差距扩大。

视角	分类	原因
理论视角	Off-policy	用旧策略 $\pi_{\theta_{old}}$ 的数据优化当前策略 π_{θ} , 通过 ratio 实现重要性采样
工程实践	Approximately on-policy	数据只复用 3-10 轮, 之后必须重新采样 (不能无限复用)
算法家族	Trust Region Method	既非严格 on-policy 也非完全 off-policy, 而是 "带约束的有限离线优化"

你的直觉完全正确:
 PPO 确实是 off-policy (用旧数据优化新策略),
 但不能像 DQN 那样无限复用数据, 所以实践中被 "近似" 视为 on-policy.

TRPO 与 PPO 都是半离线的, 一方面它确实使用了旧数据来算新分布下的值, 但注意旧和新之间一直都只差了最多一个 epoch, 并且有严格的 hard constraints 或 clip 限制分布差异.

而不像 DQN 能用缓冲区反复用更久前的数据.

对, 注意 TRPO 原始使用中还没有 epochs 的概念, 到 PPO 论文才提到

PPO - Clip Surrogate objective

$$L^{CLIP}(\theta) = \hat{E}_t [\underbrace{\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)}_{\downarrow \text{CPI}}]$$

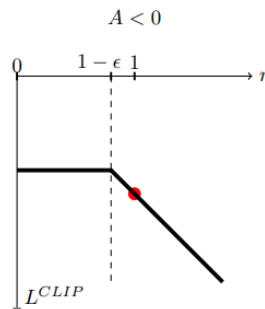
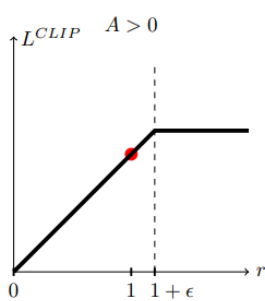
同个情况

- $\hat{A}_t > 0, \text{ratio} > 1+\epsilon$
- $\hat{A}_t < 0, \text{ratio} < 1-\epsilon$
- $\hat{A}_t > 0, \text{ratio} < 1-\epsilon$
- $\hat{A}_t < 0, \text{ratio} > 1+\epsilon$

被 clip 到 $1+\epsilon$, 防止过分更新 - PPO 中 CLIP 的梯度不考虑 (进了 CLIP 就不更新).

新策略选择好动作的概率大幅降低, 需大力补救.
新策略选择坏动作的概率大幅升高.

后两种情况中 $\min()$ 发挥作用, 采用 CPI 那一项, 可以及时补救.



PPO CLIP 的梯度传递情况: 防止过度优化, 但 $A > 0$ 但 $r \ll 1$ 或 $A < 0$ $r \gg 1$ 时不限制.

PPO - Penalty -- 自适应系数

- Using several epochs of minibatch SGD, optimize the KL-penalized objective

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right] \quad (8)$$

- Compute $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{old}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$

- If $d < d_{\text{targ}}/1.5$, $\beta \leftarrow \beta/2$

- If $d > d_{\text{targ}} \times 1.5$, $\beta \leftarrow \beta \times 2$

Algorithm.

① 联合优化

$$\text{Surrogate} = L_t^{\text{CLIP+VF+TS}}(\theta) = \hat{\mathbb{E}}_t [L_t^{\text{CLIP}}(\theta) - \underbrace{c_1 L_t^{\text{VF}}(\theta)}_{\text{减价值函数拟合的MSE}} + \underbrace{c_2 S[\pi_{\theta}](s_t)}_{\text{最大熵加强探索}}]$$

虽然这种联合优化并不是DDPG和SAC那样的深度耦合，但也起到了提高经验利用率， c_1 与 c_2 协调各网络的作用，未平衡优化目标。同时各模块的梯度仍相对独立，保有可能解释性

② Advantage Estimator

(1) n-step Estimator:

$$\hat{A}_t = -V(s_t) + r_t + \gamma V(s_{t+1}) + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V(s_T)$$

每个 rollout 长为 T (远小于 Episode length)，每次更新只需采集 rollout 个数据，平衡了方差和偏差。

(2) n-step 方法就是截断的 GAE 在 $\lambda=1$ 的时候

$$\hat{A}_t = s_t + \gamma \lambda s_{t+1} + \dots + (\gamma \lambda)^{T-t+1} s_{T-1}$$

$$s_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

→ N -parallel actors. 每次迭代收集 $N \times T$ 个数据，跑 num_epochs 次更新

Algorithm 1 PPO, Actor-Critic Style

for iteration= $1, 2, \dots$ **do**

for actor= $1, 2, \dots, N$ **do**

 Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps 采集数据.

 Compute advantage estimates $\hat{A}_1, \dots, \hat{A}_T$

end for

 Optimize surrogate L wrt θ , with K epochs and minibatch size $M \leq NT$ 对同一轮数据

$\theta_{\text{old}} \leftarrow \theta$ 跑K次梯度.

end for